Aufg.:

```
(define x 41)
(define foo (lambda (n) (+ x n)))
(foo 1)  ⇝ 42
(define bar (lambda (x) (foo 1)))
(bar 22) ⇝ 42   lexical scope
         ↘ 23   dynamic scope
```

Aufg.:

```
(define x '(1 2 3))
(list x) ⇝ ((1 2 3))

(define stronje
  (lambda (list)
    (list list)))

(stronje '(1 2 3))
(stronje list) ⇝ (<proc ..>)
(stronje (lamba (x) x))
(stronje stronje)
```

Aufg.:

$n \mapsto (n\ 2n\ 3n)$

$n,k \mapsto (n\ kn\ k^2n)$

6 Bedingte Anweisungen und Verzweigungen

Typ boolean Prädikate

Aufg.:
- zero?
- even?

$\wedge$	&&	and
$\vee$	||	or
$\neg$	!	not

Aufg.:
- odd? mithilfe von even?
- atom? (Symbol oder ganze Zahl)

```
(= a b c)
(< a b c)

(and)

(or)

(if ⌴ ⌴ ⌴)
```

Aufg.:

① abs — Betrag

② min — Minimum zweier Zahlen

③ sub1

$$n \mapsto \begin{cases} n-1 & n > 0 \\ 0 & n = 0 \end{cases}$$

```
int dist (int a, int b) {
   if (a > b)
      a - b;
   else
      b - a;
}
```

```
a > b ? a - b : b - a;
```

```
(if (even? x)
    (g (f-even x))
    (g (f-odd x)))

(g (if (even? x) (f-even x) (f-odd x)))

(g ((if (even? x) f-even f-odd) x))
```

Aufg. (which m n)

(i)	$m=0 \vee n=0$	'zero
(ii)	$m=n$	'equal
(iii)	$m \neq n$	'different

if ist Spezialoperator. Warum?

```
(if (zero? x) (+ x 2) (/ 1 x))
```

and und or sind ebenfalls Spezialoperatoren

Scheme: alles außer #f ist wahr

```
(or (< 7 6) #f (* 7 6) (= 7 6))   ⇝ 42
(and (> 7 6) #t (* 7 6))          ⇝ 42
(not (* 7 6))                     ⇝ #f
```

Aufg: distinguish:

'even	gerade Zahl
'odd	ungerade Zahl
'symbol	Symbol
'null	leere Liste
'no-idea	...

7 Funktionale Programmierung (Einschub)

Funktionen sind First-Class-Objekte

Wir (als nächste Woche) programmieren immer rein funktional!

```
(lambda (x)
  (foo x)
  (+ x 1))
```

```
(define foo
  (lamba (x)
    (if (even? (+ (* x x) 3))
        (/ (+ (* x x) 3) 2)
        (add1 (+ (* x x) 3)))))
```

```
(define foo
  (lamba (x)
    (let ((z (+ (* x x) 3)))
      (if (even? z)
          (/ z 2)
          (add1 z)))))
```

```
(define foo
  (lamba (x)
    ((lamba (z)
       (if (even? z)
           (/ z 2)
           (add1 z)))
     (+ (* x x) 3))))
```